Ubermodel

Meet Shah / $\mathrm{M}\eta\tau\ \Sigma\alpha\chi$
Independent
meetshah9625@gmail.com

22nd April, 2026

Contents

I propose an almost novel framework for training Ai. Calling it "UBERMODEL" ("oo-ba(r)-model") from Nietzsche's Ubermensch. [1]

Let there be n models with (for the sake of demonstration in the momeent) two parameters each x, y. So each model hast two parameters and they would, preferably be mututally exclusive. In real applications, this would be multiple, possibly hundreds or/of thousands, yk weights, biases, and the rest.

Let for every model $f_{model} = g(x_{model}, y_{model})$. g could be any operation on the parameters. like addition, multiplication, mean, median, mdoe, other ML-ey stuff. For the sake of demonstration, let's assume $g(x, y) = x + y \implies f = x + y$.

Let's assume for now the goal is $-((x-3)^2 + (y-2)^2)$. THis would, in real applpications, be the task they optimise for. So yes, Ubermodel is supposed to be a supervised learning method.

Let ρ is a redistribution framework. i'll do later what it may.

I may or may not use these "for the sake of demonstration"s rn but I might for examples.

So. When the training starts, each model is assigned an f. This is very important.

Then models are grouped. Based on the following formulation:

For models A and B

Group if $|f_A - f_B| < k$

Where k is either a fairly small constant ranging from like $0.1 - 0.3$ in my tests. Or it can be computed by the following

$$k = \frac{1}{n} \sum_{z=1}^{n} |f_z - \bar{f}|$$

which is pretty self-explanatroy other than maybe $\bar{f}$. So it is the average f of the all of the models. so it is just sorta itself $\frac{\sum f}{n}$. n is finite so this is doable.

After that, group all of these models. Just like using the formulae aforementioned. There maybe more than 2 or 3 or any number for that regard. You can maybe hard code the number of models in each group. If you do, keep in mind it might cause some weird edge cases.

After that, test each model in each group for the objective. Btw, if you don't have proper x and y , just use `random.random()` in python because $K \in (0.1, 0.3)$ can help with the grouping.

The comparision shown above ($|f_A - f_B| < k$) might seem that it only works for two models, which, yes. But, for more than two models, do this: that compare each model (within that group) with the f of the other model that isn't in the group. And if, for most of the models the comparision is true (ie. $|f_A - f_B| < k$), then

[1] Etymology: post-hoc rationalision of the name, Ubermodel just sounded dope at first. though to make sense of etymology other than appealing to intuition or emotion is post-hoc rationality.

have that model in the group. If the split is 50/50. Compare with nearby-f group (ie. groups with average f such that the average f of those ($\bar{f}$) satisfies $|\bar{f} - f_{\text{socially awkward model}}| < k$ where socially awkward model is the best I could come with for the model not fitting in anywhere).

PS: I am using absolute values everywhere because if in some edge case the f-difference happens to go negative, it is sorta difficult to explain, but I'm hoping you picking up what im putting down.

Having grouped all of these models, test them. Like, say for example take our demo task $-((x-3)^2 + (y-2)^2)$. This should peak around $(3, 2)$. This is somehow a 3D function so: to future me, do not try to plot this is desmos graphing calculator and spend 20 minutes debugging math; use the 3D tool.

So, assign `score` - χ based on the performance in the goal. So let's take a rough guess, say a population of 5 models with x and y as such $(1, 2), (3, 4), (5, 6), (7, 8), (9, 10)$. Their respective f's would be $3, 7, 11, 15, 19$. Let's say K based on this would be ≈ 5. so let's group them

```
Group 1: |3-7| < 5 => |-2| < 5 => Group: [3, 7]
Group 2: |11 - 15| < 5 => |-4| < 5 => Group: [11, 15]
Group 3: [19]
```

I know I did not account for the proper grouping I did not account for more than 2 models per group, but that is intentional because it would take too much time and for such a small dataset mostly everything would just fit into one group.

So let's now test these models for their tasks:

$$\text{Model 1} : (1, 2) \implies -((1-3)^2 + (2-2)^2) = -4$$

$$\text{Model 2} : (3, 4) \implies -((3-3)^2 + (4-2)^2) = -4$$

$$\text{Model 3} : (5, 6) \implies -((5-3)^2 + (6-2)^2) = -20$$

$$\text{Model 4} : (7, 8) \implies -((7-3)^2 + (8-2)^2) = -52$$

$$\text{Model 5} : (9, 10) \implies -((9-3)^2 + (10-2)^2) = -100$$

Now model 1 and 2 performed the best here. Again, very small population so difficult to show, in reality this would be more..yk...sciency. For example this:

```
Models with f values:
  Model 0: [0.8625082795671632, 0.2146319551482777], f = 1.0771
  Model 1: [0.8417095719082842, 0.050102100873433325], f = 0.8918
  Model 2: [0.33822165567066753, 0.6087483153608314], f = 0.9470
  Model 3: [0.5987903599235507, 0.20369709897157684], f = 0.8025
  Model 4: [0.49959971920564517, 0.9250281545091644], f = 1.4246
  Model 5: [0.35406470637510756, 0.10010256418209063], f = 0.4542
  Model 6: [0.5206780790616988, 0.5721247597711773], f = 1.0928
  Model 7: [0.9007134551118084, 0.20229718703441446], f = 1.1030
  Model 8: [0.63272005943212, 0.6456831025201051], f = 1.2784
  Model 9: [0.09407922606732366, 0.164392908649447], f = 0.2585
  Model 10: [0.04230669927990116, 0.7585694285146086], f = 0.8009
  Model 11: [0.8150011523610914, 0.3420442710919325], f = 1.1570
  Model 12: [0.3214878689297178, 0.9492023896495395], f = 1.2707
  Model 13: [0.6033277323973542, 0.9393438370004683], f = 1.5427
  Model 14: [0.16280211023671043, 0.14981211540791417], f = 0.3126
  Model 15: [0.35512047179589346, 0.8939856142454359], f = 1.2491
  Model 16: [0.6675325963039088, 0.023161172734672886], f = 0.6907
  Model 17: [0.07374406483409823, 0.9860299973460317], f = 1.0598
  Model 18: [0.8614512289678202, 0.08348585917028584], f = 0.9449
  Model 19: [0.8036655236950527, 0.47618688014202715], f = 1.2799

Groups (K = 0.2734):
  Group 0: [[0.09407922606732366, 0.164392908649447],
            [0.16280211023671043, 0.14981211540791417],
            [0.35406470637510756, 0.10010256418209063]]
```

```
  Group 1: [[0.6675325963039088, 0.023161172734672886],
            [0.04230669927990116, 0.7585694285146086],
            [0.5987903599235507, 0.20369709897157684],
            [0.8417095719082842, 0.05010210087343325],
            [0.8614512289678202, 0.08348585917028584],
            [0.33822165567066753, 0.6087483153608314]]
  Group 2: [[0.07374406483409823, 0.9860299973460317],
            [0.8625082795671632, 0.2146319551482777],
            [0.5206780790616988, 0.5721247597711773],
            [0.9007134551118084, 0.20229718703441446],
            [0.8150011523610914, 0.3420442710919325],
            [0.35512047179589346, 0.8939856142454359],
            [0.3214878689297178, 0.9492023896495395],
            [0.63272005943212, 0.6456831025201051],
            [0.8036655236950527, 0.476186880142027 15]]
  Group 3: [[0.49959971920564517, 0.9250281545091644],
            [0.6033277323973542, 0.9393438370004683]]

Scores:
  Best Model: [0.6033277323973542, 0.9393438370004683], Score = -6.8690
End of cycle----------------------------
```

(from `logs.txt`).

So having done all this, all in all, we have compared each model with their score (i won't use the logs.txt one, only the population=5 things). We see that models 1 and 2 performed equally. But throw out the examples for now.

So each model that performs the best in each group, call it winner or samubermodelos[2].

The top 3-4 (or any number, according to the programmer) shall be considdered. Again, I used a very small population here, in real world the population should be at minimum 150-200. So consider the top few models.

Till now, most of this was standard machine learning.

But now, the loser models. They are deleted. but not meaninglessly. Their f is redistributed. This is where ρ comes in.

Each of the loser models are killed. Their f gets redistributed among the winners of the group.

Maybe. for now lets say L is the set of losers and W is the set of winners; for group-wise (which is preferable) L_A and W_A for group A.

So

$$\text{Total Loser } f = \sum_{a=1}^{|L_A|} f_a$$

Then

$$\forall w \in W$$

$$f_w = f_w + \alpha \cdot (\frac{\text{Total Loser } f}{|W|})$$

Where α is a small constant to control the strength of redistribution

I know $f_w = f_w + ...$ is sort of mathemtatical black magic but to implement this would be along the lines of `f_w += ...`. I need α because you do need a scalar for redistribution because this is sometimes it maybe that $(\frac{\text{Total Loser } f}{|W|})$ equaluates to 1 or 0, so just, all in all, α is necessary to control "how hard" to redistribute.

And then since f is updated. Remember $g(x, y)$? So if f is updated, the x and y should be updated too. So this happens through identifying the best performing model (ie. argmax $\chi(n)$) over the elapsed time (t maybe instead of time using cycles, iterations). This will be the guide for evolution.

[2] Etymology: "sam-" is Old-English for "semi" and "ubermodel" is pretty self-explanatory and the "-os" comes from Nominative singular case of Greek and Latin.

Then each model shall update its parameters to move towards the best. For x and y this would be

$$x_i = x_i + \eta(x_{best} - x_i) y_i = y_i + \eta(y_{best} - y_i)$$

(again same math black magic `x_i += \eta(...)` and `y_i += \eta(...)`)

where, self evidentally $(x_{best} - x_i)$ and $(y_{best} - y_i)$ are the direction towards which to evolve (increase or decrease).

$\eta \in (0, 1)$ is a step size. As in how aggressively move towards the goal while preserving diversity (if η is close to 0). The models converge more quickly if η is close to 1. So we shall keep it between them to ensure proper behaviour.

Then decrease the population, killing the losers.

This process continues until only one model remains. Ie. more and more iterattions with updating f, executing ρ, and updating k as need be.

And that, is....BEHOLD, AN UBERMODEL.

Some anticipated questions

- If diversity spikes later, then what?
  - As in, what if in later cycles, where population is low, then if the models evolve some good trait then what? Like if its last few cycles and say, for image recognition model, it very well recognises an image. The issue here is sort of difficult to explain without proper terms. So the question is mainly just what if in late cycles, diversity in the model spikes?
    - This would be resolved by increasing k momentarily and all in restoring conditions to such that momentarily the diversity can spread out, if only it is beneficial. This I can maybe account for later properly and mathematically, but for now this is the idea.
- Why Ubermodel rather than other algorithms?
  - Because, I have not tested it yet, but it might outperform some algorithms; or be novel; or something else. I do not yet have any evidence for why this would be better, for I have yet to implement this, if ever.
- The choice of g decides everything much, there are endless operation on any number of parameters. How can you gaurantee the best?
  - I would say Ubermodel is not something to be run in isolation. Maybe you can run multiple Ubermodels with different choices of g and then at the end compare all of them to create what could be maybe Offeraubermodelos[3]. Maybe you can even get the previous question (why ubermodel) answer by comparing many different algorithms to Ubermodel.
- Is this not reducing models too aggressively?
  - Perhaps. But you can choose different k, α and η for different speeds.

[3] "offera-" (Old-English: "up", "above") "ubermodel" "-os" (Nominative singular noun ending in Greek/Latin)